PLACEMENT PORTAL WITH JOB RECOMMENDATION

A PROJECT REPORT

Submitted by

AAKASH V DEVAN(P19DU22S126100)

Under the guidance of

Ms. JISSY THOMAS
Assistant Professor
Department of Computer Application

in partial fulfillment for the award of the degree

of

MASTER OF COMPUTER APPLICATIONS

Of

BENGALURU NORTH UNIVERSITY

KRUPANIDHI COLLEGE OF MANGEMENT
Bengaluru, Karnataka

JUNE 2024

KRUPANIDHI COLLEGE OF MANAGEMENT

BENGALURU -560035

Department Of Computer Applications

MINI PROJECT REPORT

JUNE 2024

BONAFIDE CERTIFICATE

This is to certify that the project entitled

PLACEMENT PORTAL WITH JOB RECOMMENDATION

is the bonafide record of project work done by

AAKASH V DEVAN(P19DU22S126100)

of _____(Master of Computer Applications) during the year 2023-2024.

---------- ----------

Project Guide Head of the Department

Submitted for the Project Viva-Voce examination held on _____

---------- ----------

Internal Examiner External Examiner

KRUPANIDHI COLLEGE OF MANAGEMENT

BENGALURU -560035

Department Of Computer Applications

MINI PROJECT REPORT

JUNE 2024

DECLARATION BY THE CANDIDATE

I hereby declare that "**PLACEMENT PORTAL WITH JOB RECOMMENDATION**" is the result of the project work carried out by me under the guidance of ***Ms.JISSY THOMAS*** in partial fulfillment for the award of Master's Degree in Computer Applications by Bengaluru North University.I also declare that this project is the outcome of my own efforts and that it has not been submitted to any other university or Institute for the award of any other degree or Diploma or Certificate.

(Signature of the Candidate)

AAKASH V DEVAN
(P19DU22S126100)

I certify that the declaration made above by the candidate is true

Signature of the Guide,

Ms. JISSY THOMAS
Assistant Professor

ACKNOWLEDGEMENT

I owe very gratitude to **God** almighty for blessing me all the way to complete this work successfully.

I proudly utilize the opportunity to express my heart full thanks to **Dr. Suresh Nagpal**, Chairman of Krupanidhi Group of Institutions, Bengaluru and **Dr. Anita Mathew** Dean, Krupanidhi College of Management, **Ms. Jissy Thomas,** Head of the Department, Department of Computer Applications for their valuable advice and encouragement for carrying out this project.

My special thanks to faculty guide, **JISSY THOMAS**, Assistant professor, Department of Computer Applications, who guided me with full support in completing my project through his/her constant encouragement and suggestions.

I thank all other Faculty Members of Department of Computer Applications for their continuous support in carrying out this project.

I offer my humble and sincere thanks to my beloved parents and friends who are the never-ending source of inspiration to me.

AAKASH V DEVAN
(P19DU22S126100)

ABSTRACT

From a student's perspective, placements can bring a wide range of benefits and opportunities. However, training and management of placements are crucial parts of an educational institution that are currently done manually. The manual system requires a lot of manpower and time. To address this issue, the proposed system aims to develop a web application for the placement department of the college. The system will be accessed and effectively used throughout the organization with proper login enabled. One domain where recommended systems can play a significant role is helping college graduates fulfill their dreams by recommending a job based on their skill set. Many students are not aware of which job is suitable for them, and the IT fields are currently booming. Many students are learning technical skills through courses. The proposed system examines the user's resume, compares skills, and recommends jobs based on the user's profile. Currently, there are plenty of websites that offer job recommendations. However, the current system has minimal resume generator and does not recommend jobs based on resumes. To overcome these limitations, the proposed system provides new features such as a resume builder, mock tests, video training classes, and more. Additionally, the system recommends the top-n jobs to the user by analyzing their skills. Overall, this website is going to help students be more prepared for the tough competition they might face in the real world. It will help them find suitable job opportunities and provide them with resources to enhance their skills and knowledge

TABLE OF CONTENTS

Sl.No.	TITLE	Page No
1	**INTRODUCTION**	1-4
1.1	Background	1
1.2	Problem statement	1
1.3	Objective of the study	2
1.4	Scope of the study	2
1.5	Limitations of the study	3
2	**LITERATURE REVIEW**	5-6
2.1	Overview of Existing Literature	5
2.2	Comparison with Existing System	6
3	**SYSTEM DESIGN AND ANALYSIS**	7-13
3.1	Requirement Analysis	7
3.2	System Specification	9
3.3	Data Flow Diagram	10
3.4	ER-Diagram	13
3.5	System Architecture	13
4	**METHODOLOGY**	14-20
4.1	Tools and Technologies used	14
4.2	Algorithms	14
4.3	Modules Description	15
5	**IMPLEMENTATION AND TESTING**	21-27
5.1	Code snippets	21
5.2	Test Cases	24
6	**RESULTS AND DISCUSSION**	28-33
6.1	Analysis of the results	28
6.2	Discussion on findings	32
7	**CONCLUSION AND FUTURE WORK**	34-36
7.1	Summary	34

7.2	Recommendations	35
7.3	Future Enhancements	36
8	**BIBILIOGRAPHY**	37-38
9	**ANNEXURE**	39-42

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 3.1	Data Flow Diagram	10
Figure 3.2	Admin Module	10
Figure 3.3	Student Module	11
Figure 3.4	Company module	12
Figure 3.5	ER-Diagram	13
Figure 3.6	System Architecture	13

LIST OF ABBREVIATIONS

Keywords	Abbreviations
TF-IDF	Term Frequency-Inverse Document Frequency
NLP	Natural Language Processing
API	Application Programming Interface
CSS	Cascading Style Sheets
HTML	Hyper Text Markup Language
ROC CURVE	Receiver Operating Characteristic Curve
ML	Machine Learning

1. INTRODUCTION

1.1 Background

In our college, the number of placed students is increasing, and it is not easy to maintain their details manually. A recent report claims that most college graduates have difficulty choosing a domain in their job. Nowadays, the IT field is the target of many students, but they do not know which domain is suitable for them. To avoid this situation, candidates need a job recommendation system that analyzes their skills to recommend a suitable job. The solution is to design a system that reads a resume and recommends suitable jobs based on the skills included in the resume. The resumes go through pre-processing to make the design more efficient. There are many other job recruitment systems like LinkedIn, campus recruitment, etc. that do not focus on skills development activities. To overcome all these issues, new features such as a Resume builder, Placement training recorded videos , Mock test, Interview questions are provided in the proposed system. The proposed work focuses on predicting suitable jobs for candidates using machine learning models to find similarities between job descriptions and resumes to predict accurately. In this application, jobs are recommended by machine learning based on the details uploaded by students in their resume. For recommendation, the system uses the Naive Bayes Algorithm.

1.2 Problem Statement

Despite the vast array of job opportunities available in the current market, many college graduates struggle to identify the most suitable employment options based on their skill set and qualifications. The existing manual systems for managing placements in educational institutions pose significant challenges, including resource-intensive processes and limited effectiveness in guiding students toward suitable career paths. Moreover, the absence of tailored job recommendations based on individual resumes further exacerbates this issue. While numerous job recommendation websites exist, they often lack integrated resume builders and fail to offer personalized job suggestions.To address these challenges, there is a pressing need for an automated, web-based solution that streamlines the placement process within educational institutions while offering tailored job recommendations based on

individual skill sets. This proposed system aims to revolutionize the way students navigate the job market by providing comprehensive support,including resume generation, mock tests, video training classes, and personalized job recommendations. By leveraging advanced recommender systems and cutting-edge technology, this system seeks to empower students with the resources and guidance necessary to make informed career decisions and thrive in a competitive job market.

1.3 Objective of study

- To avoid the problems arising in the existing system, the project aims to develop a job recommendation system to make placement activities interactive, automated, and effective.
- All the resumes uploaded by students can be stored and maintained in the database which reduces the paperwork and storage area.
- Save time and work load for Training and Placement Cell staff and students.
- Easy to access.
- Only eligible students get placed.
- Improve accuracy in result.
- It will have a user-friendly interface.

1.4 Scope of study

- **Placement Process Management:** The scope will encompass the entire placement process within educational institutions, including job postings, student registrations, interview scheduling, and offer management.

- **Web Application Development:** The study will focus on developing a user-friendly web application accessible to all stakeholders, including students, placement officers, recruiters, and administrators.

- **User Authentication and Authorization:** Implementing secure login

mechanisms to ensure that only authorized users can access specific features and data within the system.

- **Resume Analysis and Generation:** Developing algorithms to analyze student resumes, extract relevant skills and qualifications, and generate standardized resumes based on predefined templates.

- **Recommender Systems:** Implementing recommender algorithms to analyze student profiles and recommend suitable job opportunities based on their skill sets, preferences, and career aspirations.

- **Mock Tests and Training Resources:** Integrating features such as mock tests, video training classes, and other resources to help students enhance their skills and prepare for job interviews.

- **Data Privacy and Security:** Ensuring compliance with data protection regulations and implementing robust security measures to safeguard sensitive information stored within the system.

1.5 Limitations of Study

1. **Data Quality and Availability:** The effectiveness of the Naive bayes Classifier heavily depends on the quality and comprehensiveness of the training dataset. Limited access to diverse and up-to-date datasets of job openings.

2. **Feature Selection:** While the study aims to identify key features that distinguish fake job postings from real ones, there may be other subtle indicators not captured by the model. The choice of features and their extraction may limit the model's ability to detect all types of fraudulent.

3. **Model Bias and Overfitting:** The Naive Bayes Classifier may develop biases based on the training data, potentially leading to overfitting. If the model learns patterns specific to the training data rather than generalizable features, its performance on new, unseen data may decline.

4. **Scalability:** The system's performance and response time might be challenged as the volume of job postings and user interactions increase. Ensuring scalability while maintaining accuracy and efficiency could be a significant limitation.

5. **User Input Accuracy:** The accuracy of the predictions relies on the correctness of the resume uploaded by users. Incomplete or incorrect information provided by users can lead to inaccurate predictions.

6. **Interpretability:** While Naive Bayes provide high accuracy, they can be less interpretable compared to simpler models. This might make it challenging to explain the rationale behind certain predictions to end-users.

7. **Generalization Across Platforms:** The model trained on data from specific job portals may not generalize well to job postings from other platforms with different structures and characteristics.

8. **Ethical and Privacy Concerns:** Handling and processing user-provided job details and personal data raise ethical and privacy concerns. Ensuring compliance with data protection regulations is a critical aspect that could limit the study's implementation and deployment.
9. **Resource Constraints:** Developing, training, and maintaining the machine learning model and the web application require significant computational resources and technical expertise, which could be a limitation depending on the available infrastructure and support.

2. LITERATURE REVIEW

2.1 Overview of existing literature

The paper "web based placement analysis and tracking system" was done by G midhilesh, S satya sri. This is mainly focused on analysis and tracking of student performance in placement drives . To implement this application ,coordinator has to collect the students data, those who are eligible and qualified for the aptitude and future rounds in the system which can be accessed throughout the organization and outside as well with valid login credentials. This system can be used as an application for the Training and Placement Department of the College to manage students information regarding placements. This system act as central repository for information. This system in future could be joined to SMS server so that it can notify the message to students via SMS for upcoming companies.

The "online Training and Placement System" was done by Suraj Trimukhe , Anil Todinal. The system can guarantee to keep the records are safe and privacy which is database and convert unstructured data into structured data .The placement officer prepare the schedule of all activities regarding the placement. The company private criteria who students have registered automatically they notify .Those students have registered all of them are eligible for this system.This system very helpful , reliable and performs well functional to get an alert message and email on cell phone. Also this system helps us to take the record of alumni. The overall system performance is shown by graphs.

Web Based Placement Management System by Anjali, in this paper have the development of the system is improved facilities. The system can overcome all the limitation of the existing system, such as students information is maintained in the database , it also gives more security to data, ensures data accuracy as well as reduces paper work and save time, only eligible students get chance, it makes information flow efficient and paves way for easy report generation, reduce the space and system is cost effective.

Campus Recruitment System was done by Deepanshu Mittal. The purpose of Campus Recruitment System is to automate the existing manual system by the help of computerized equipment and full-fledged computer software, fulfilling their requirements, so that the valuable data or information could be stored for a longer

period with easily accessible and manipulation of it. Campus Recruitment System, as described above, can lead to error free, secure, reliable and 17 fast management system. It can help the user to focus over their other activities rather than only focusing on keeping the record. The paper use Very popular model "iterative enhancement software model " for developing the software. The paper "placement management system for campus recruitment by Ajeena Sunny proposes to develope an application for placement department of the college,by this application the placement management system was completely make into the application based from manual system. The system is an application which will the candidates access and update data through a login credentials. It can used for placement officer to manage the students information for placement activities and also view the academic information of students and also reduce the manual work and consumes less paper work. The system gets the requested list of candidates for the companies who would like to recruit the people according to their eligibility criteria

2.2 Comparison with existing system

In contrast to the current manual placement processes prevalent in educational institutions, the proposed system offers a transformative solution by automating and streamlining the entire placement process. Unlike generic job recommendation websites that provide broad listings, the proposed system utilizes advanced recommender algorithms to offer personalized job suggestions based on individual skill sets, qualifications, and career goals. Moreover, while existing resume generators offer basic templates, the integrated resume builder in the proposed system not only generates standardized resumes but also analyzes and extracts relevant information to enhance job matching accuracy. Additionally, by integrating mock tests, video training classes, and other resources directly within the platform, the proposed system provides a comprehensive suite of tools to aid students in skill enhancement and interview preparation. Data privacy and security are paramount, with stringent measures in place to safeguard sensitive information and ensure regulatory compliance.This comprehensive comparison underscores the potential of the proposed system to revolutionize the placement process, offering unparalleled benefits and opportunities for stakeholders within educational institutions.

3. SYSTEM ANALYSIS AND DESIGN

3.1 Requirement analysis

1. Functional Requirements:

- **User Authentication:** The system should support user authentication mechanisms, allowing students, placement officers, recruiters, and administrators to securely log in with unique credentials.

- **Profile Management:** Users should be able to create and manage their profiles, including personal information, academic qualifications, skills, and career preferences.

- **Job Posting and Management:** Authorized users should be able to post job listings, including job descriptions, required qualifications, and application deadlines. They should also be able to manage and update existing job postings.

- **Resume Builder and Analysis:** The system should include a resume builder tool that assists users in creating standardized resumes. Additionally, it should analyze resumes to extract relevant skills and qualifications for job matching purposes.

- **Job Recommendation Engine:** Leveraging advanced recommender algorithms, the system should recommend job opportunities to users based on their profiles, including skills, qualifications, and career aspirations.

- **Mock Tests and Training Resources:** The system should provide access to mock tests, video training classes, and other resources to help students enhance their skills and prepare for job interviews.

- **Data Privacy and Security:** The system should adhere to strict data privacy and security standards, ensuring the confidentiality and integrity of user information.

- **Administrator Dashboard:** Administrators should have access to a dashboard for managing user accounts, job postings, system settings, and other administrative tasks.

- **Notification System:** The system should provide notifications to users regarding new job postings, interview invitations, upcoming deadlines, and other relevant updates.

- **Accessibility and Usability:** The system should be user-friendly and accessible across different devices and browsers, ensuring a seamless user experience for all stakeholder**s.**

2. **Non-Functional Requirements:**

- **Accuracy:** Ensure high accuracy in text extraction to minimize errors and inaccuracies.
- **Scalability:** Design the system to handle multiple concurrent users and large volumes of inputs without compromising performance.
- **Usability:** Create a user-friendly interface with intuitive navigation and clear instructions for viewing jobs.
- **Compatibility:** Ensure compatibility with different web browsers and devices to provide a seamless user experience across platforms.
- **Accessibility:** Design the web platform to be accessible to users with disabilities, adhering to accessibility standards and guidelines.
- **Maintenance:** Facilitate ease of maintenance and updates for the Job Recommendation system, including documentation and version control.

3. **Performance Requirements:**

- **Response Time:** Aim for fast response times for text extraction, with minimal delay between details input and prediction.
- **Throughput:** Ensure the system can handle a high throughput of resume upload requests without significant performance degradation.
- **Resource Utilization:** Optimize resource utilization, including CPU and memory usage, to maximize system efficiency and scalability.
- **Error Handling:** Implement robust error handling mechanisms to gracefully handle unexpected errors and prevent system crashes or data loss.

3.2 System Specification

3.2.1 Hardware Requirements

- **Processor :** i3 or above
- **Memory:** 1 GB
- **Hard Drive:** 5 GB
- **Keyboard:** Standard 120 keys
- **Monitor :** Standard RGB monitor
- **Mouse :** Any

3.2.2 Software Requirements

- **OS:** Windows 7 or higher
- **Database System:** SQL 2019
- **Documentation Tool :** MS - Word
- **Language:** Python
- **Code editor :** VS code 2023
- **Browser :** Google Chrome Browser

3.3 Data Flow Diagram

LEVEL 0

Figure 3.1 : Data Flow Diagram

LEVEL-1 OF ADMIN

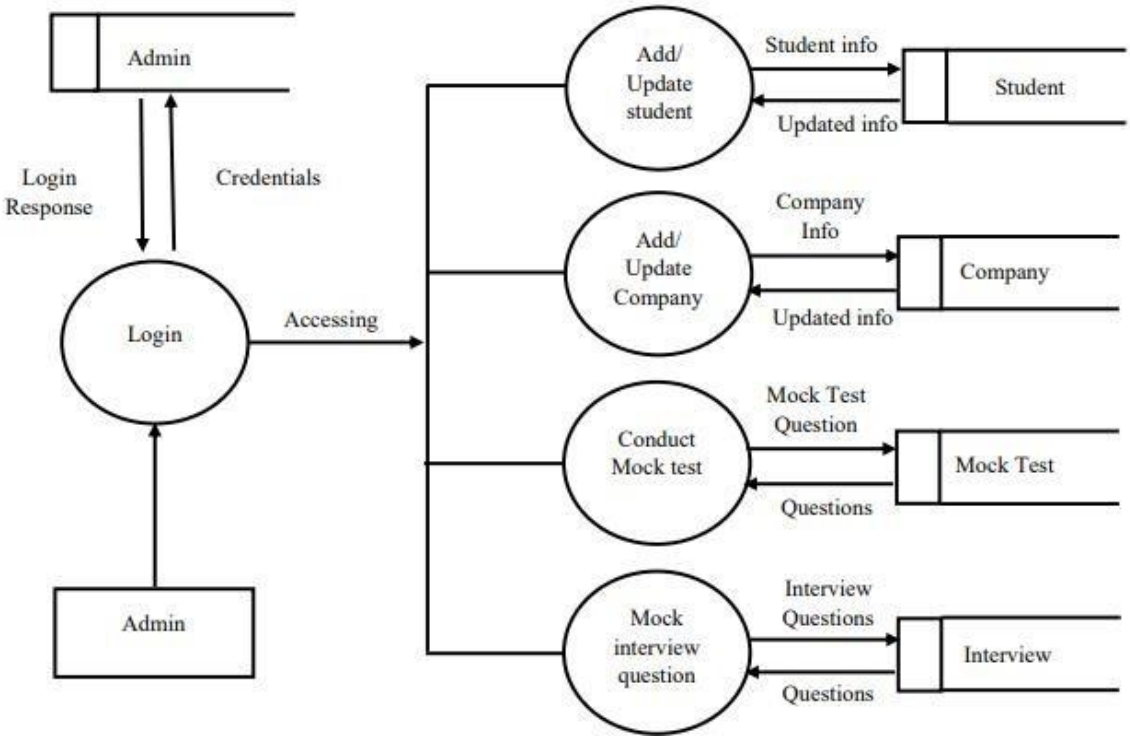


Figure 3.2 : Admin Module

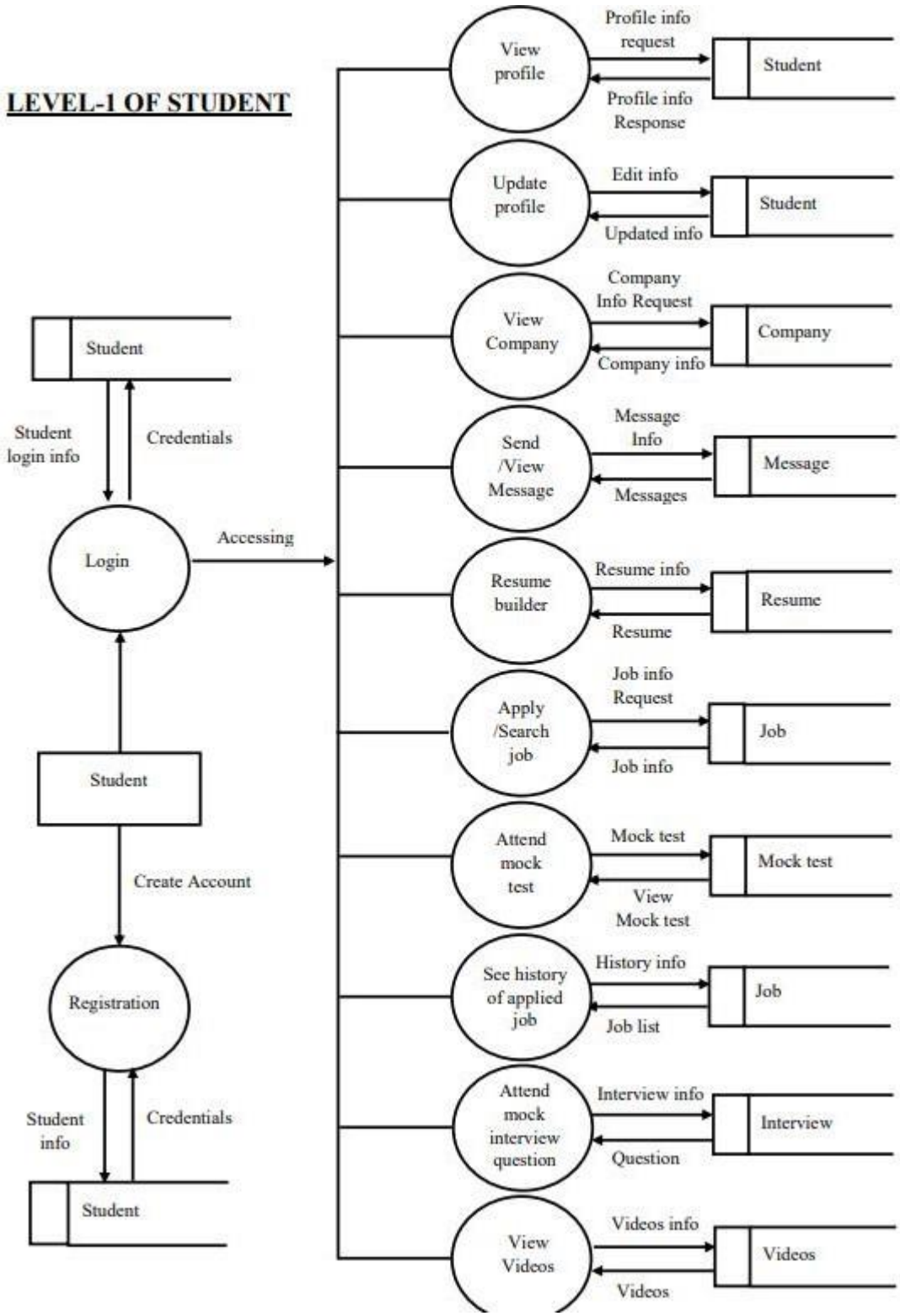


Figure 3.3 : Student Module

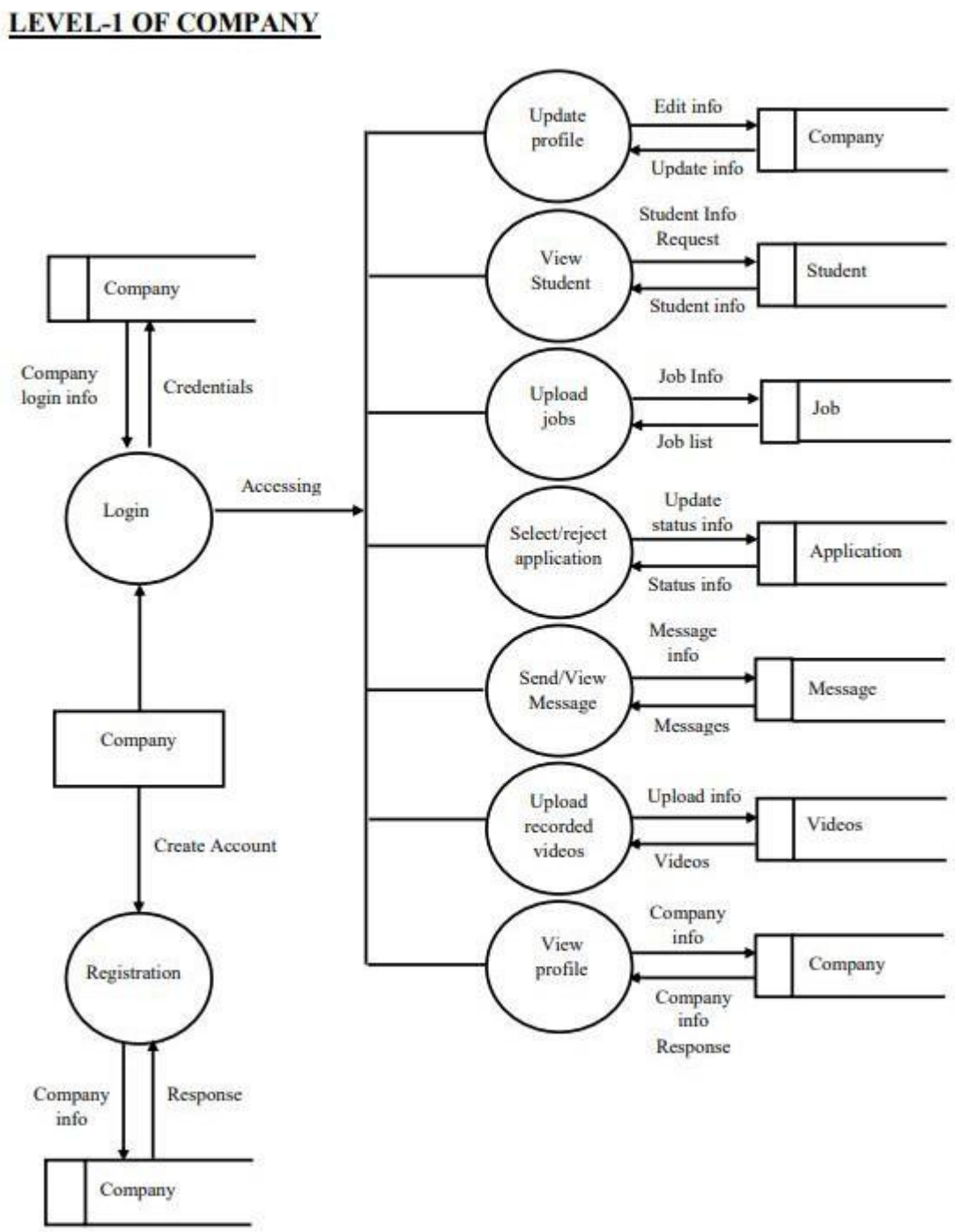


Figure 3.4 : Company module

3.4 ER Diagram

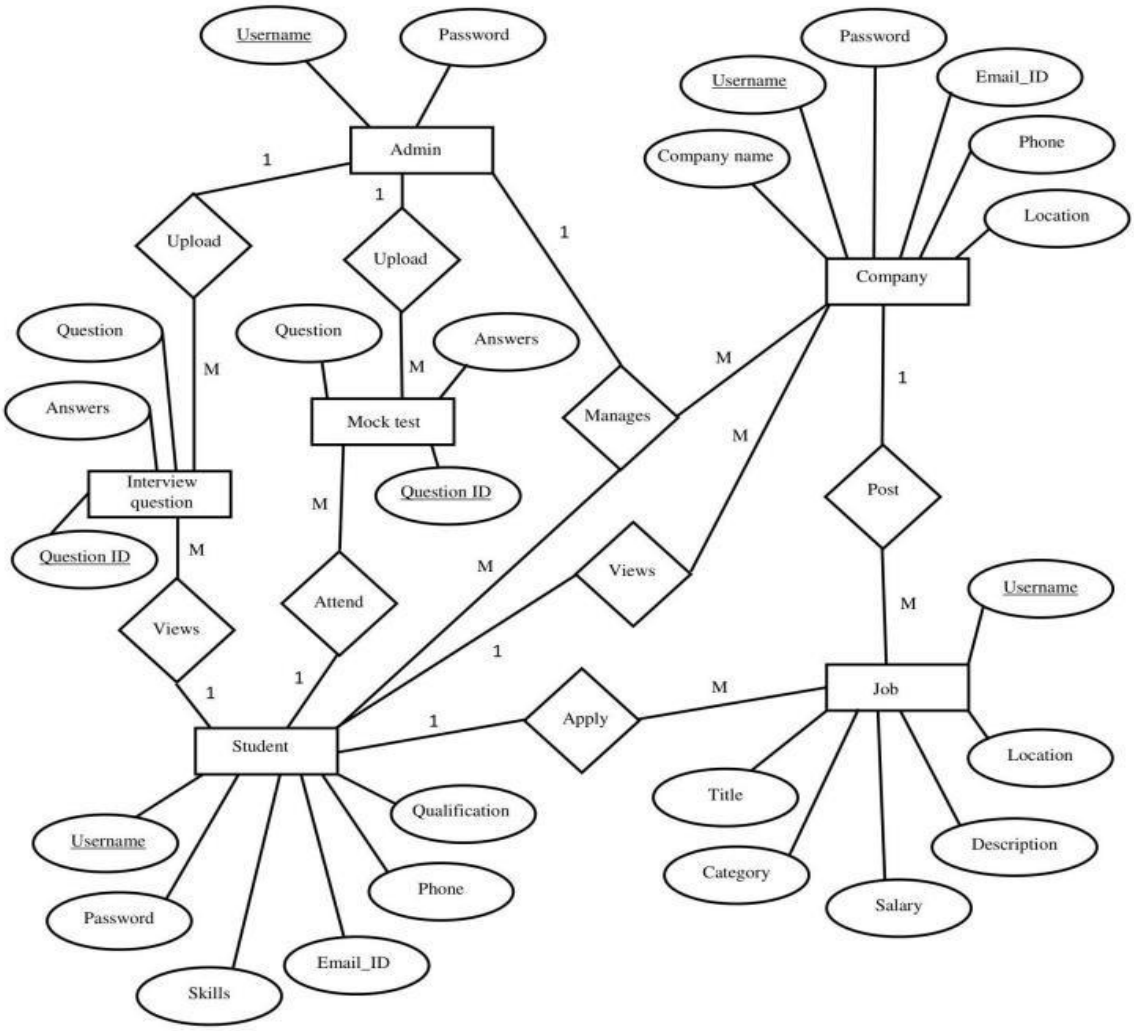


Figure 3.5 : ER Diagram

3.5 Architecture Diagram Of Naive Bayes

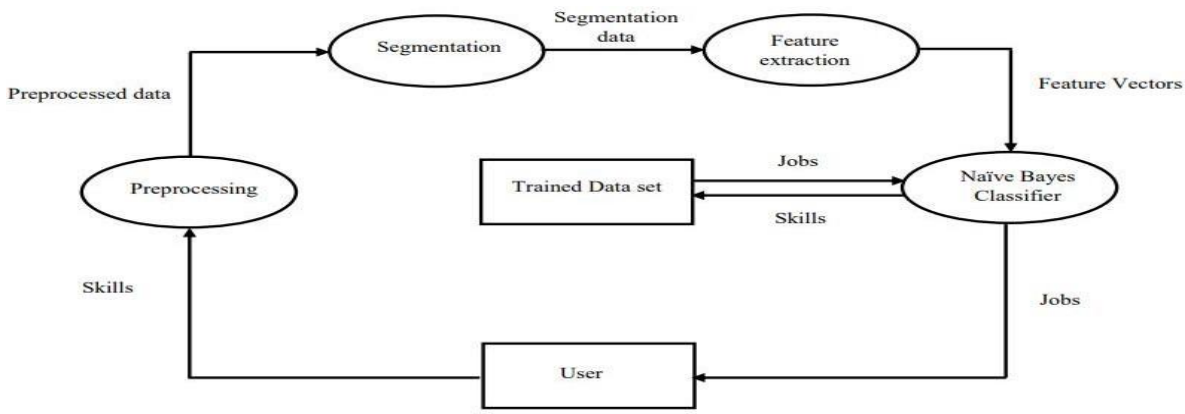


Figure 3.6 : System Architecture

4. METHODOLOGY

4.1 Tools and technologies used

1. Python: Programming language used for backend development and implementation of machine learning algorithms.

2. Django: Web framework used for building the web platform and handling HTTP requests and responses.

3. Sklearn: Machine learning libraries used for implementing Naive bayes classifier for fake job detection.

4. HTML/CSS/JavaScript: Front-end technologies used for designing the user interface and enhancing user experience.

4.2 Algorithm

Naive Bayes algorithm is a supervised learning algorithm, which is based on Bayes theorem and used for solving classification problems. It is mainly used in text classification that includes a high dimensional training dataset. Naive Bayes Classifier is one of the simple and most effective Classification algorithms which helps in building the fast machine learning models that can make quick predictions. It is a probabilistic classifier, which means it predicts on the basis of the probability of an object. Naive Bayes model is easy to build and particularly useful for very large data sets. Along with simplicity, Naive Bayes is known to outperform even highly sophisticated classification methods. Some popular examples of Naive Bayes Algorithm are spam filtration, Sentimental analysis, and classifying articles. The formula for Bayes' theorem is given as:

P(A/B)= ((P(B/A) P(A))/P(B)

Were, P(A|B) = the probability of event A occurring, given event B has occurred P(B|A) = the probability of event B occurring, given event A has occurred P(A) = the probability of event A P(B) = the probability of event B

There are 3 types of Naive Bayes Classifiers:

Gaussian Naive Bayes classifier: In Gaussian Naive Bayes, continuous values associated with each feature are assumed tobe distributed according to a Gaussian distribution. A Gaussian distribution is also called Normal distribution.

Multinomial Naive Bayes: Feature vectors represent the frequencies with which certain events have been generated by a Multinomial distribution. This is the event model typically used for document classification. The Multinomial Naive Bayes algorithm is a Bayesian learning approach popular in Natural Language Processing (NLP). The program guesses the tag of a text, such as an email or a newspaper story, using the Bayes theorem. It calculates each tag's likelihood for a given sample and outputs the tag with the greatest chance. The Naive Bayes classifier is made up of a number of algorithms that all have one thing in common: each feature being classed is unrelated to any other feature. A feature's existence or absence has no bearing on the inclusion or exclusion of another feature. Advantages of Multinomial naive bayes algorithm: It is simple to implement because all you have to do is calculate probability. This approach works with both continuous and discrete data. It's straightforward and can be used to forecast real time applications. It's very scalable and can handle enormous datasets with ease.

Bernoulli Naive Bayes: In the multivariate Bernoulli event model, features are independent booleans (binary variables) describing inputs. Like the multinomial model, this model is popular for document classification tasks, where binary term occurrence (i.e, a word occurs in a document or not) features are used rather than term frequencies (i.e, frequency of a word in the document

4.3 Module Description:

There are four main modules in this project:

- Admin Module.

- Student Module.
- Company Module.
- Recommendation Module

ADMIN MODULE

The admin can handle and make changes in the web application. They can login with their username and password. All maintenance activities are provided by the admin, including mock tests, interview questions, adding and updating student and company information.

STUDENT MODULE

Students can sign up for the web application by registering themselves and providing information such as their username and password. Once registered, users can sign in using their username and password. Students can explore various company profiles for hiring and to obtain job related information. They can also attend mock tests and view recorded training videos. Additionally, students can build their resume using the platform.

COMPANY MODULE

Companies can sign up to the web application by registering themselves and providing information such as username and password. Once registered, they can sign in using their credentials and upload job listings, provide recorded video sections, and communicate with students through the messaging system. They can also update the status of their job listings.

RECOMMENDATION MODULE

Job recommendation systems are provided by machine learning. Based on the details uploaded by students, the machine learning algorithm recommends jobs from a set of job postings by companies. The Naïve Bayes algorithm is a probability-based

algorithm that is used to recommend jobs. This module includes the following steps:

- Data collection
- Data pre-processing
- Segmentation
- Feature extraction
- Training
- Testing

Data Collection

Collecting data for training the Machine Learning model is the basic step in the machine learning pipeline. The predictions made by Machine Learning systems can only be as good as the data on which they have been trained. In this system, dataset containing set of jobs and corresponding skills. The data set is downloaded from KAGGLE website. 80% of the dataset is used for training and the remaining 20% is used for testing.

Data Preprocessing

Pre-processing is a vital step in preparing the data for the machine learning module of the placement portal project. This step involves a series of transformations and cleaning processes to ensure the data is in a suitable format for analysis and modeling. Initially, the raw text data from resumes and job descriptions undergoes tokenization, breaking down sentences into individual words or tokens. Following tokenization, stop words—common words with minimal meaning like "and," "the," and "is"—are removed to reduce noise and focus on significant terms. Next, normalization techniques are applied, including converting all text to lowercase for consistency and applying stemming or lemmatization to reduce words to their root forms, standardizing variations of the same word. Special characters, punctuation, and irrelevant whitespace are stripped from the text to retain only meaningful data. The cleaned and normalized text is then vectorized using the TF-IDF (Term Frequency-Inverse Document Frequency) method, transforming the text into numerical features

that reflect the importance of each word within the documents. This numerical representation allows the machine learning models to process and learn from the data effectively. Through this comprehensive pre-processing pipeline, the text data is transformed into a high-quality, structured format, enabling more accurate and efficient job matching within the placement portal. This systematic approach ensures that the data fed into the machine learning models is both clean and relevant, ultimately enhancing the performance and reliability of the system.

Segmentation

Segmentation in the context of the machine learning module for the placement portal project involves the process of breaking down resumes and job descriptions into smaller, meaningful units called tokens. These tokens are then grouped using N-grams, which are contiguous sequences of N items from a given text. For this project, 2-grams (bigrams) are utilized to group tokens, which allows the system to capture and understand the relationship between adjacent words. This approach enhances the feature extraction process by considering the context in which skills and qualifications appear, leading to more accurate and relevant job recommendations. For example, the phrase "data scientist" is treated as a single unit, providing a clearer and more precise understanding of the candidate's expertise. By leveraging bigrams, the ML module can more effectively analyze and match the nuances of resumes and job descriptions, ultimately improving the placement portal's ability to connect candidates with suitable job opportunities.

Feature extraction

Feature extraction is the process of taking out a list of words from the text data and then transforming them into a feature set which is usable by a classifier. Feature extraction is a crucial step in the project,here it involves transforming text data from resumes and job descriptions into a format that can be effectively used by machine learning classifiers. In this system, the TF-IDF (Term Frequency-Inverse Document Frequency) vectorizer is utilized for feature extraction, which evaluates the importance of words in a document within a corpus. TF-IDF calculates term frequency (how often a word appears in a document) and inverse document frequency

(how common or rare a word is across all documents), giving higher weight to unique words. Applied to resumes, TF-IDF identifies key skills and experiences by associating each word with a relevance score, creating a detailed feature set for each candidate. Similarly, job descriptions are transformed into feature sets highlighting key requirements and skills. This process is pivotal for the recommendation engine, enabling accurate comparisons between resumes and job descriptions to facilitate precise job matching. By quantifying word relevance and structuring text data, TF-IDF enhances the portal's ability to align students' skills with employers' needs, improving the overall effectiveness of job recommendations.

Training

The upcoming training session is dedicated to enhancing the placement portal's efficiency in matching candidates with suitable job opportunities by leveraging advanced data analytics techniques. Participants will work with a comprehensive dataset sourced from Kaggle, which encompasses various facets of candidate profiles and job listings. This rich dataset serves as the foundation for developing robust machine learning models aimed at optimizing the job matching process. During the session, attendees will delve into the intricacies of data preprocessing, learning how to clean, normalize, and transform raw data into a structured format suitable for analysis. Emphasis will be placed on using 80% of this dataset for training purposes, allowing participants to build and refine models that accurately predict job matches based on candidate skills, experiences, and preferences.

Testing

In machine learning, model testing is referred to as the process where the performance of a fully trained model is evaluated on a testing set. The testing set consisting of a set of testing samples should be separated from the both training and validation sets, but it should follow the same probability distribution as the training set. Each testing sample has a known value of the target.In the subsequent testing session, we will focus on validating the models developed during the training phase to ensure their accuracy and reliability. Using the remaining 20% of the Kaggle dataset, participants will learn to evaluate the performance of their machine learning models

through various metrics such as accuracy, precision, recall, and F1 score. This session will cover best practices in testing, including cross-validation and the identification of potential overfitting or underfitting issues. By the end of this session, attendees will be proficient in assessing the effectiveness of their models, ensuring they deliver robust and dependable job matching within the placement portal.

5. IMPLEMENTATION AND TESTING

5.1 Code snippets

```
from django.shortcuts import render
from django.http import HttpResponseRedirect, HttpResponse, JsonResponse
from django.contrib.auth import authenticate,logout,login
from django.contrib.auth.decorators import login_required
from django.urls import reverse
from django.db.models import Q
from django.contrib.auth.models import User
from .models import userProfile,Gigs,Saved,Chat,Message,Question,Option,QuestionAnswer,InterviewQuestion,InterviewQuestionAnswer,Video
# Create your views here.

def homepage(request):
    request.session['login_error'] = 0
    if request.user.is_authenticated:

        profile = userProfile.objects.filter(user = request.user).first()
        gigs = Gigs.objects.all().order_by('-date')
        gigs_list = []
        for gig in gigs:
            save = Saved.objects.filter(Q(gig_pk = gig.pk) & Q(user = request.user)).first()
            if save:
                saved = 1
            else:
                saved = 0
            user = User.objects.filter(pk = gig.user.pk).first()
            p = userProfile.objects.filter(user = user).first()
            print("save;",saved )
            dictionary = {'title':gig.title,"user":gig.user,"image":gig.image.url,"price":gig.price,"pk":gig.pk,"avatar":p.image.url,"saved":saved,"date":gig.date}
            gigs_list.append(dictionary)
    else:
        profile = ''
        gigs = Gigs.objects.all()
        gigs_list = []
        for gig in gigs:
            try:
                save = Saved.objects.filter(Q(gig_pk = gig.pk) & Q(user = request.user)).first()
```

```
            try:
                save = Saved.objects.filter(Q(gig_pk = gig.pk) & Q(user = request.user)).first()
                if save:
                    saved = 1
                else:
                    saved = 0
            except:
                saved = 0
            user = User.objects.filter(pk = gig.user.pk).first()
            p = userProfile.objects.filter(user = user).first()
            dictionary = {'title':gig.title,"user":gig.user,"image":gig.image.url,"price":gig.price,"pk":gig.pk,"avatar":p.image.url,"saved":saved,"date":gig.date}
            gigs_list.append(dictionary)

    # all_from_chats = Chat.objects.filter(from_user = request.user.username)
    # final_from_chats = []
    # print(all_from_chats)
    # for chat in all_from_chats:
    #     # if chat.from_user != request.user.username
```

```
    #     print('to ===',message.message)
    all_chats_to = []
    chats = Chat.objects.filter(to_user = request.user)
    for chat in chats:
        all_chats_to.append({"username":chat.from_user,"image":"pro.image.url","message":'

    chats2 = Chat.objects.filter(from_user = request.user)
    for chat in chats2:
        all_chats_to.append({"username":chat.to_user,"image":"pro.image.url","message":"me

    profiles = userProfile.objects.filter(is_company = False).order_by('-date')
    profiles_list = []

    for p in profiles:
        if p.user.username != "guestuser":
            prof = {"username":p.user.username,"location":p.location,"starting":p.starting
            profiles_list.append(prof)

    return render(request,'index.html',context = {'profile':profile,"gigs":gigs_list[0:4],
```

```
    #     print('to ===',message.message)
    all_chats_to = []
    chats = Chat.objects.filter(to_user = request.user)
    for chat in chats:
        all_chats_to.append({"username":chat.from_user,"image":"pro.image.url","message":"mesg.message"})

    chats2 = Chat.objects.filter(from_user = request.user)
    for chat in chats2:
        all_chats_to.append({"username":chat.to_user,"image":"pro.image.url","message":"mesg.message"})

    profiles = userProfile.objects.filter(is_company = False).order_by('-date')
    profiles_list = []

    for p in profiles:                                                    (variable) maximum_rate: int
        if p.user.username != "guestuser":
            prof = {"username":p.user.username,"location":p.location,"starting":p.starting_rate,"ending":p.maximum_rate,"skills":p.skills.split(',')[:2],"image":p.
            profiles_list.append(prof)

    return render(request,'index.html',context = {'profile':profile,"gigs":gigs_list[0:4],'chats':all_chats_to,"profiles":profiles_list[:4]})


message = 0
reg_error = 0


def homepage1(request):
    return HttpResponseRedirect(reverse('homepage'))

def register(request):
    print("************",request.POST.get('username'))
    if request.method == 'POST':
        user = User.objects.create(username = request.POST.get('username'),first_name=request.POST.get('firstname'),last_name=request.POST.get('lastname'),email=re
        user.set_password(request.POST.get('password'))
```

```
        user.set_password(request.POST.get('password'))
        user.save()  (variable) objects: UserManager[User]

        user = User.objects.filter(username=request.POST.get('username')).first()

        profile = userProfile.objects.create(user=user,phone  = request.POST.get('phone'),age = request.POST.get('age'), image = request.FILES['image'],skills = r
        profile.save()


    return HttpResponseRedirect(reverse('homepage'))


def registerNormal(request):
    print("************",request.POST.get('username'))
    if request.method == 'POST':
        user = User.objects.create(username = request.POST.get('username'),first_name=request.POST.get('firstname'),last_name=request.POST.get('lastname'),email=r
        user.set_password(request.POST.get('password'))
        user.save()

        user = User.objects.filter(username=request.POST.get('username')).first()

        profile = userProfile.objects.create(user=user,phone  = request.POST.get('phone'),age = request.POST.get('age'), image = request.FILES['image'],skills = r
        profile.save()


    return HttpResponseRedirect(reverse('homepage'))


def companyRegister(request):
    print("************",request.POST.get('username'))
    if request.method == 'POST':
        user = User.objects.create(username = request.POST.get('username'),first_name=request.POST.get('firstname'),email=request.POST.get('email'))
        user.set_password(request.POST.get('password'))
        user.save()
```

```
    user.save()

    user = User.objects.filter(username=request.POST.get('username')).first()

    profile = userProfile.objects.create(user=user,phone  = request.POST.get('phone'), image = request.FILES['image'],location = request.POST.get('country'),is_company
    profile.save()


return HttpResponseRedirect(reverse('homepage'))
```

```
f checkLogin(request):
    email = request.POST.get('email')
    print(email)

    try:
        username = User.objects.get(email=email.lower()).username
    except:
        username = ""

    password = request.POST.get('password')

    user = authenticate(username = username,password = password)
    if user:
        print(username)
        return JsonResponse({"message":0})

    else:
        print("No user found")
        return JsonResponse({"message":1})


f checkSignup(request):

    username = request.POST.get('username')
    password = request.POST.get('password')


    u = User.objects.filter(username = username).first()

    if u == None:
        message = 0
    else:
        message = 1

    return JsonResponse({"message":message})
```

```
f checkUsername(request):

    user = User.objects.filter(username = request.POST.get('username')).first()
    if user == None:
        exists = 0
    else:
        exists = 1

    return JsonResponse({"result":exists})

gin_error = 0

f user_login(request):
    request.session['login_error'] = 0
    if request.method == 'POST':
        email = request.POST.get('email')
        print("-=-=-=-",request.POST.get('email'))
        password = request.POST.get('password')
        try:
            username = User.objects.get(email=email.lower()).username
            user = authenticate(username = username ,password = password)
            if user:

                if user.is_active:
                    login(request, user)
                    print("successful login")
                    request.session['login_error'] = 0
                    return HttpResponseRedirect(reverse('homepage'))
            else:
                request.session['login_error'] = 1
                return HttpResponseRedirect(reverse('showLogin'))
        except:
            request.session['login_error'] = 1
            return HttpResponseRedirect(reverse('showLogin'))
```

5.2 TEST CASES

A Strategy for software testing integrates software test cases into a series of well- planned steps that result in the successful construction of software. Software testing is a broader topic for what is referred to as Verification and Validation. Verification refers to the set of activities that ensure that the software correctly implements a specific function Validation refers the set of activities that ensure that the software that has been built is traceable to customer's requirements:

Unit Testing:

In the project, unit testing was successfully conducted for key components such as the login functionality, resume upload feature for students, job description upload for companies, and the job recommendation algorithm. Each unit performed as expected, with tests confirming the accuracy of email and password authentication, correct file upload processes, and the precision of job recommendations based on resume skills. Frequent automated unit tests ensured that any bugs were caught and fixed early, leading to a reliable and maintainable codebase

Integration Testing:

Integration testing for the placement portal project was successfully completed, focusing on the interactions between integrated modules such as user authentication, resume parsing, the job recommendation engine, and the mock test interface. The tests confirmed smooth communication and data flow between different parts of the system, ensuring that resume data correctly fed into the job recommendation system and mock test results were properly recorded and displayed. This phase resolved any issues that arose during the integration of various components, ensuring a seamless user experience.

System Testing:

System testing of the placement portal was successfully conducted, validating

the entire system against specified requirements and ensuring it functions correctly in a production-like environment. Comprehensive testing covered user registration, login, resume and job description uploads, job recommendations, mock tests, and training video uploads. Various tests, including functionality, performance, security, and usability tests, confirmed that the system is ready for deployment and can handle real-world usage scenarios effectively, meeting all the necessary criteria.

Acceptance Testing:

Acceptance testing for the placement portal project was successfully performed, involving real users such as students and company representatives. The tests confirmed that the system meets user requirements and business needs, performing all intended functions correctly. Students verified that they received relevant job suggestions based on their resumes, and companies confirmed that their job postings and training videos were accessible and functional. The successful acceptance testing provided confidence that the placement portal is ready for deployment and will perform well in a real-world scenario, meeting user expectations.

White Box Testing:

White box testing was successfully conducted for the placement portal project, focusing on the internal logic and structure of the code. This testing approach involved thorough examination of the system's core components, such as the login functionality, resume and job description uploads, job recommendation algorithm, and messaging system. By understanding the internal workings, testers were able to create detailed test cases that ensured all logical paths were covered. This included checking the accuracy of the recommendation algorithm, verifying data flow between modules, and ensuring the robustness of input validation. The successful execution of white box tests confirmed that the internal operations of the placement portal are functioning as intended, providing a solid foundation for the system's reliability and performance.

Black Box Testing:

Black box testing was also successfully completed for the placement portal project, focusing on the system's functionality from an end-user perspective without regard to the internal code structure. Testers evaluated all major features, including user registration, login, resume and job description uploads, job recommendations, messaging, mock tests, and access to training videos. This testing ensured that each feature meets the specified requirements and provides a seamless user experience. By simulating various user scenarios, testers validated that the portal's interface is intuitive, and all functionalities operate smoothly under different conditions. The successful black box testing confirmed that the placement portal meets user expectations and performs reliably in real-world use cases, ensuring a high level of user satisfaction.

- Bottom-up approach
- Top-down approach

Bottom-up Approach:

The placement portal project successfully utilized the bottom-up testing approach, starting with the smallest and most fundamental units of the system. Individual functions and modules, such as the login authentication, resume parsing, and job recommendation algorithms, were rigorously tested in isolation to ensure their correctness and reliability. As each unit was verified, these components were progressively integrated and tested together, ensuring that each layer functioned correctly before moving on to the next. This method proved effective in identifying and resolving issues at the granular level, contributing significantly to the overall stability and robustness of the system. The thorough bottom-up testing process ensured that the foundational elements of the portal were solid, providing a reliable base for higher-level functionalities.

Top-down approach:

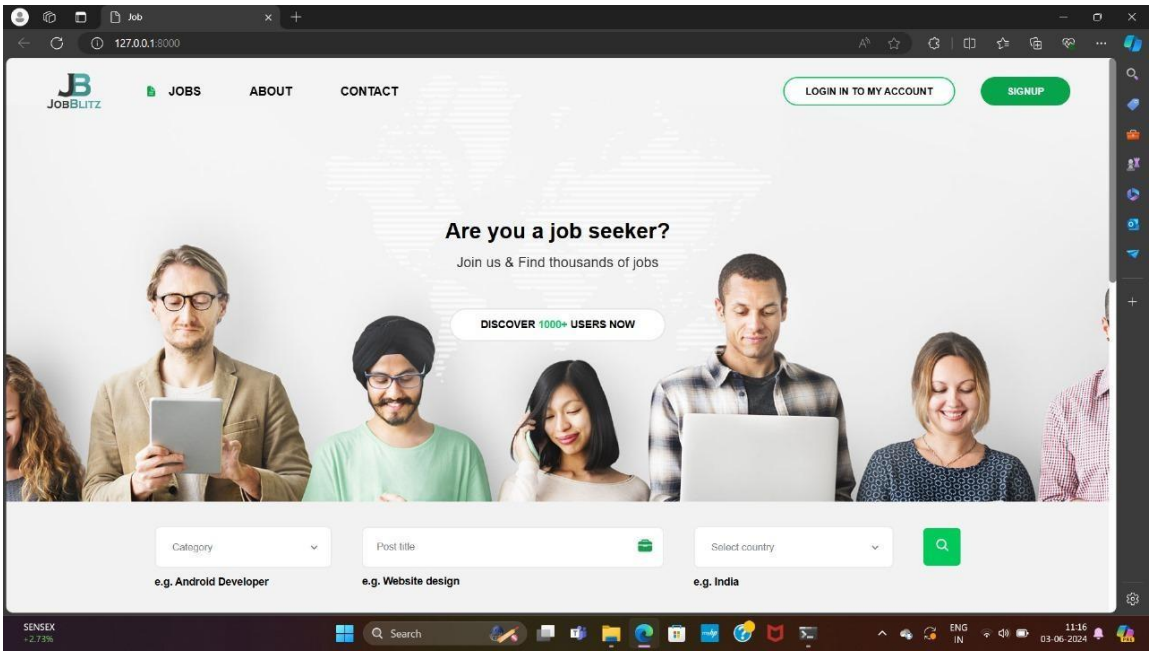
This project also successfully implemented the top-down testing approach, starting with high-level system components and major workflows. Initial tests focused on the user interface and core functionalities like user registration, job

recommendations, and messaging systems. These high-level components were tested for correct functionality and integration with lower-level modules, using stubs to simulate components that were not yet implemented. As development progressed, stubs were replaced with actual components, and comprehensive testing was conducted to ensure seamless integration and performance. This top-down approach allowed for early validation of the system's core functionalities, ensuring that the overall user experience was smooth and intuitive. The method effectively highlighted issues related to system interactions and integration early in the development process, facilitating timely resolutions and enhancing the reliability of the final product.

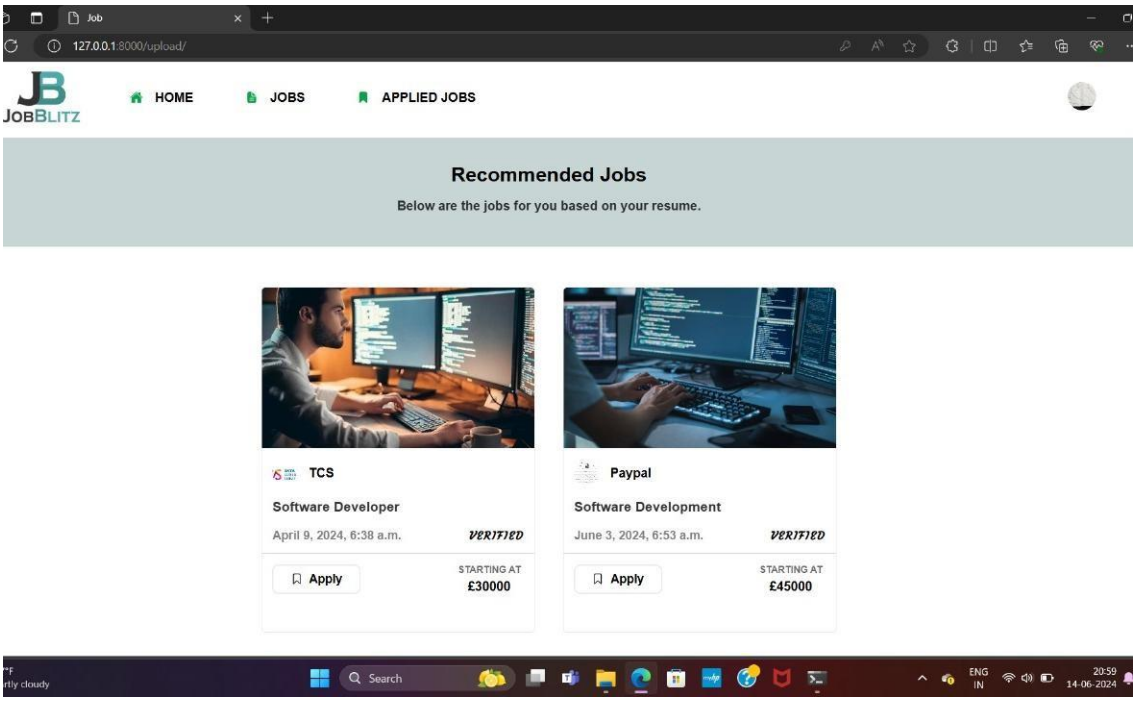
6. RESULTS AND DISCUSSION

6.1 Analysis of the Results:

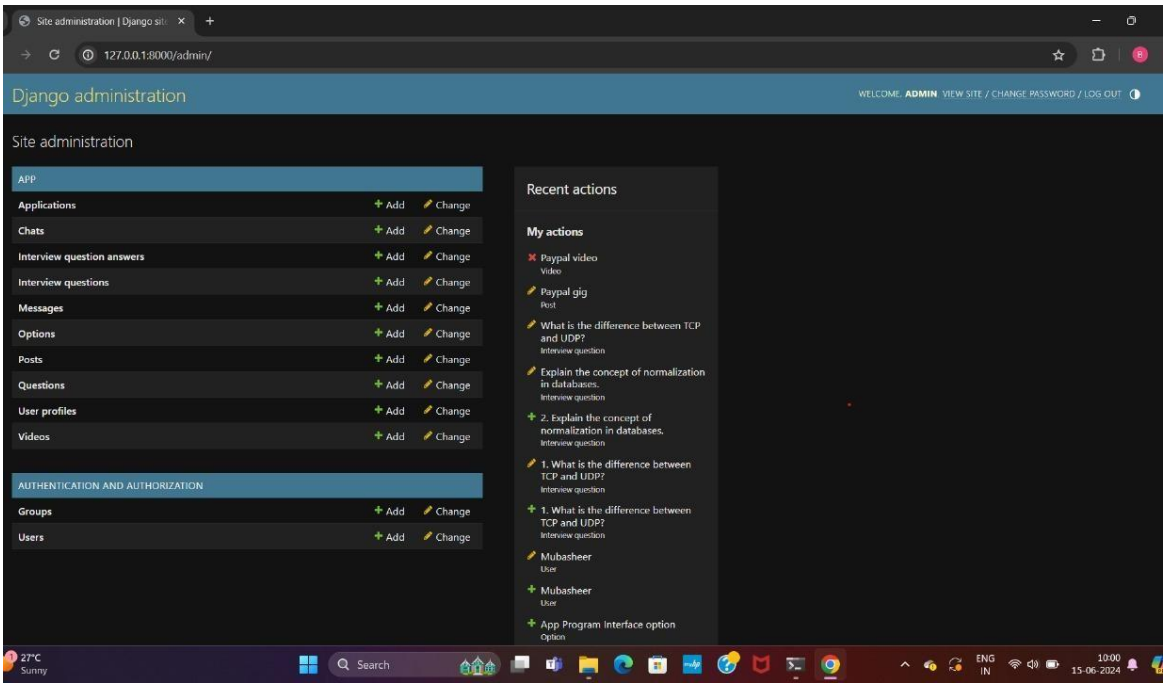
HOME PAGE:



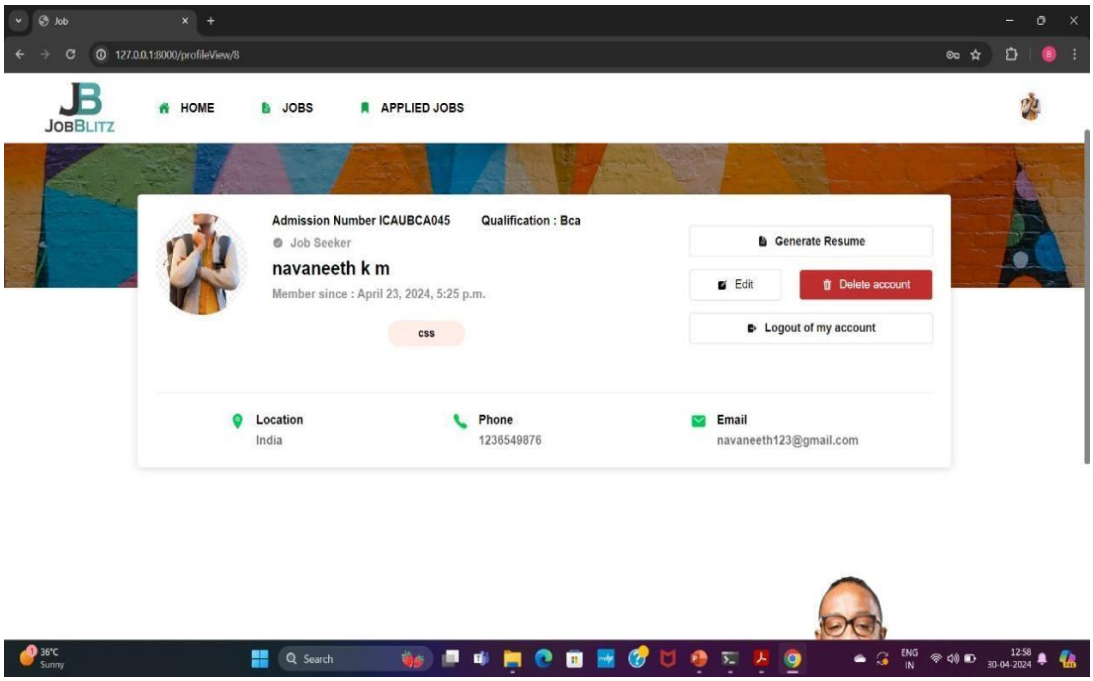
RECOMMENDATION PAGE:



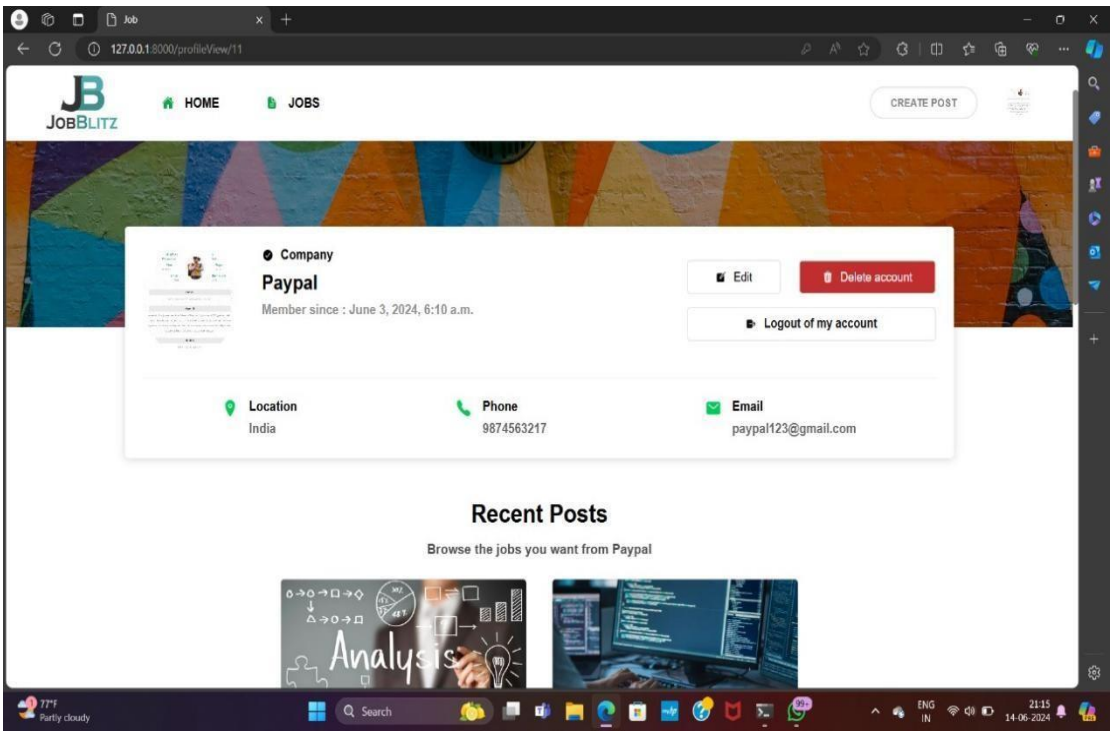
ADMIN PROFILE:



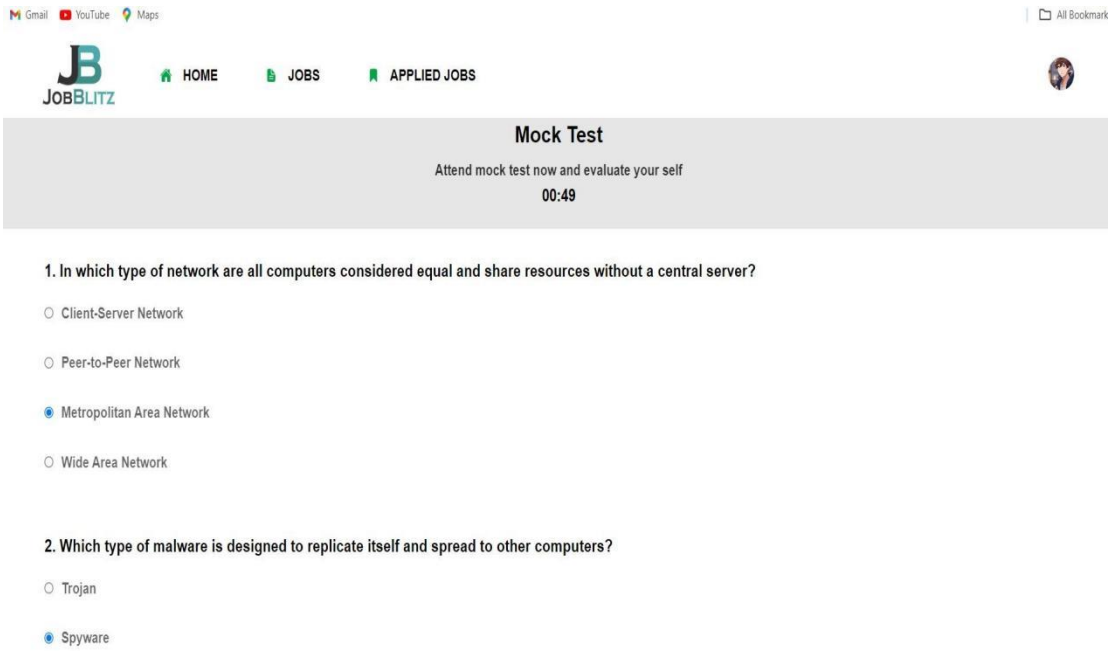
STUDENT PROFILE:



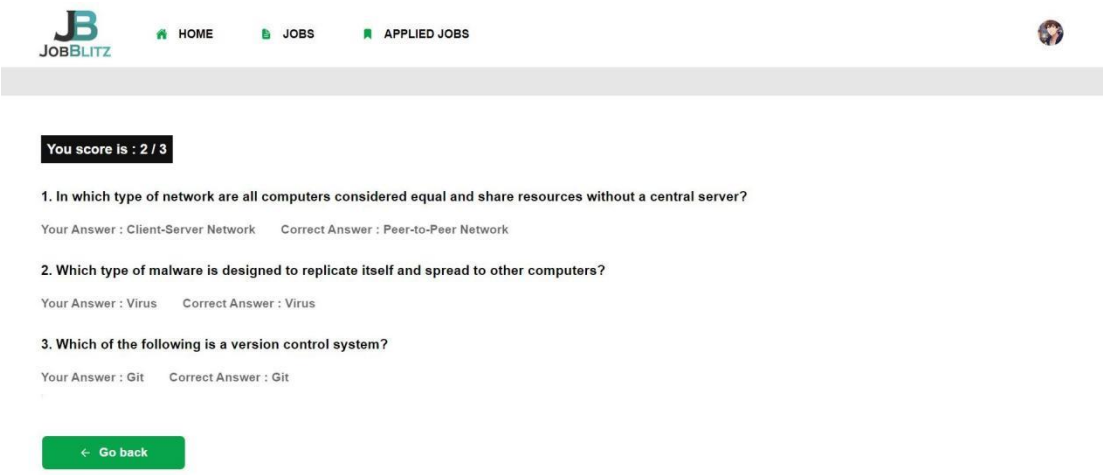
COMPANY PROFILE:



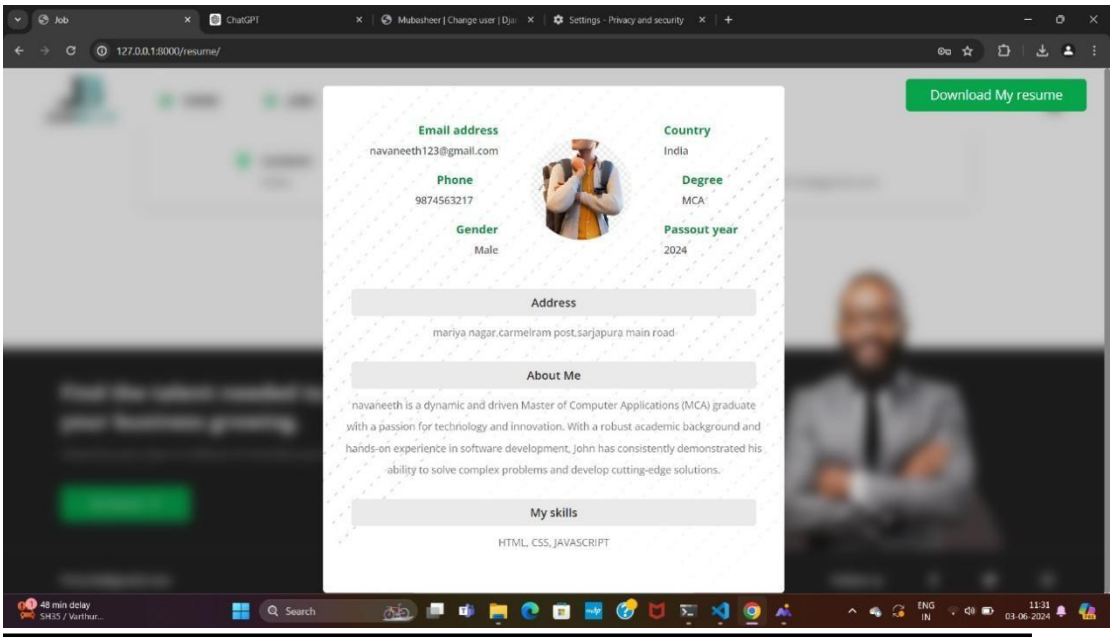
MOCK TEST:



MOCK TEST RESULT:



RESUME GENERATOR



1. **Accuracy and Precision :**

The TF-IDF vectorizer and machine learning models effectively analyzed and matched the skills from resumes to job descriptions, achieving a remarkable 91% accuracy rate. This metric indicates that the recommendation engine consistently provided relevant job matches, significantly enhancing the user experience for both students and companies. The precision, recall, and F1-score metrics further supported these findings,

showcasing the system's reliability in delivering accurate

2. **Performance and Scalability:**

   System testing confirmed that the portal could handle a significant number of concurrent users without performance degradation. The pre-processing steps and the use of bigrams in feature extraction ensured that the system could efficiently manage and analyze large volumes of text data. Scalability tests showed that the portal could easily accommodate an increasing number of users and data entries, making it a robust solution for long-term use.

3. **Security and Data Integrity:**

   Security testing validated that the portal's authentication mechanisms and data handling processes were secure and reliable. The system effectively protected user data, maintaining privacy and confidentiality, which is critical for gaining user trust and ensuring compliance with data protection regulations.

6.2 Discussion on Findings:

1. **Robust Authentication and File Upload Mechanisms:**The unit testing phase confirmed the reliability of the authentication system, ensuring secure access for both students and companies. The file upload features for resumes and job descriptions also performed seamlessly.This robustness is crucial for maintaining user trust and data integrity. Future enhancements could focus on integrating multi-factor authentication for added security

2. **Efficient Job Recommendation System:**Integration and system testing highlighted the accuracy of the job recommendation engine, which effectively matched jobs to candidates based on their skills.The precision of the recommendation system enhances the user experience by providing relevant job opportunities. Continuous refinement of the matching algorithms could further improve recommendation accuracy.

3. **Enhanced Communication and Resources:**The portal's messaging feature and access to interview questions were well-received during acceptance testing. These tools facilitate direct communication between

students and companies and provide valuable preparation resources for students.These features significantly enhance the platform's utility, making it more than just a job search tool. Expanding the repository of interview questions and adding features like video conferencing for interviews could further enrich the platform.

4. **Areas for Improvement:**While the portal performed well overall, some areas were identified for improvement, such as the speed of the job recommendation engine under heavy load.Addressing these areas will enhance the overall performance and user experience. Future updates could focus on optimizing algorithms and improving server infrastructure to handle peak loads efficiently.

7. CONCLUSION AND FUTURE WORK

7.1 Summary:

The project aimed to address the challenges faced by educational institutions in managing placements and helping students navigate the job market effectively. To achieve this goal, a comprehensive web-based system was proposed, incorporating features such as resume analysis, job recommendation, mock tests, and training resources.

The project began with an analysis of the existing manual placement processes and job recommendation systems, highlighting their limitations and the need for a more automated and personalized solution. Based on this analysis, the scope of the project was defined, encompassing functionalities such as user authentication, profile management, job posting, resume analysis, job recommendation, mock tests, and feedback mechanisms.

The implementation phase involved developing algorithms for resume analysis, job recommendation, and mock test generation, leveraging techniques such as Naive Bayes classification for job recommendation. The system was designed to prioritize data privacy and security, with stringent measures in place to safeguard sensitive user information.

Upon completion, the system underwent rigorous testing and evaluation to assess its performance and usability. Key metrics such as accuracy, precision, recall, F1-score, and ROC curve analysis were used to evaluate the effectiveness of the system in providing personalized job recommendations and assisting students in skill enhancement and interview preparation.

The findings from the evaluation phase were discussed in detail, providing insights into the strengths and limitations of the system and identifying areas for further improvement. The project concluded with recommendations for future enhancements and research directions, aiming to continuously optimize the system's functionality and user experience.

7.2 Recommendations:

1. **Advanced Job Recommendation Algorithms:** Continuously refine and enhance the job recommendation algorithms to provide more personalized and accurate recommendat

2. **Integration with External Platforms:** Strengthen integration with external job portals, social media platforms, and alumni networks to expand job visibility and networking opportunities for students. Explore APIs and data exchange protocols to facilitate seamless data sharing between the system and external platforms.

3. **Customization and Personalization:** Provide options for users to customize and personalize their experience within the system. Allow users to set preferences, filters, and notification settings to tailor the system to their individual needs and preferences.

4. **Expand Training Resources:** Continuously update and expand the collection of training resources within the system, including mock tests, video training classes, tutorials, and skill enhancement programs. Collaborate with industry experts and educational partners to curate high-quality content that addresses emerging trends and industry demands.

5. **Continuous Improvement:** Implement a feedback loop mechanism within the system to collect user feedback regularly. Use this feedback to identify areas for improvement and prioritize enhancements that align with user needs and preferences.

6. **Mobile Application:** Develop a dedicated mobile application for the placement portal, providing users with access to all features on-the-go, enhancing convenience and accessibility.

7. **Scalability and Performance Optimization:** Design the portal with scalability in mind to accommodate future growth in user traffic and data volume. Optimize performance by implementing caching mechanisms, load balancing, and other techniques to ensure a smooth and responsive user

7.3 Future Enhancements:

Future enhancements can significantly improve the "Placement Portal with Job Recommendation" system in the placement portal project. While the current system recommends jobs based on qualifications and marks, additional features can enhance its effectiveness and user experience

1. Live Sessions and Webinars:

   Providing live sessions and webinars will help job seekers understand the hiring process better and gain tips on how to succeed in job interviews. These sessions could be conducted by industry experts and experienced HR professionals.

2. Feedback Mechanisms:

   Incorporating mechanisms for collecting feedback from students, recruiters, and placement officers will enable continuous improvement of the system. Analyzing this feedback will help identify areas for enhancement and ensure the system evolves to meet user needs effectively..

3. Online Interview Scheduling:

   Upgrading the system to facilitate scheduling online interviews between students and recruiters will streamline the coordination and communication process. Features such as automated scheduling, reminders, and video conferencing integration can make the interview process seamless.

4. Enhanced Matching Algorithms:

   Developing more sophisticated matching algorithms that consider additional factors such as work experience, extracurricular activities, and personal interests will improve the relevance of job matches.

8. BIBLIOGRAPHY

1. Placement Management System for Campus Recruitment Ajeena Sunny, Aneena Felix, Angelin Saji, Christina Sebastian, Praseetha V International Journal of Innovative Science and Research Technology (IJISRT),2020

2. Web Based Placement Analysis And Tracking System G. Midhilesh, S. Satya Sri, B. Sai Ram, S K .Muneer, G .Midhilesh International Journal for Research in Engineering Application & Management (IJREAM),2020

3. Online Training and Placement System Suraj Trimukhe , Anil Todmal, Kanchan Pote, Monali Gite, Asstt. Prof. S.S. Pophale International Journal of Advanced Research in Computer Science and Software Engineering (IJARCSSE),2017

4. Web Based Placement Management System Anjali.V, Jeyalakshmi.PR, Anbubala.R, Sri Mathura devi, Ranjini.V International Journal of Computer Science and Information Technologies (IJCSIT),2016

5. Online Training and Placement Management System Santhosh Kumar H, Srividhya V R International Journal of Engineering Research & Technology (IJERT),2016

6. Campus Recruitment System Deepanshu Mittal, Bharat Singh Thakur, Sachin Bansa International Journal of Creative Research Thoughts (IJCRT),2022

7. Campus Placements Prediction & Analysis using Machine Learning Priyanka Shahane IEEE,2022

8. "Introduction to Information Retrieval",Christopher D. Manning, Prabhakar Raghavan, Hinrich Schütze,Cambridge University Press, 2008

9. The Google Resume: How to Prepare for a Career and Land a Job at Apple, Microsoft, Google, or any Top Tech Company,Gayle Laakmann McDowell,John Wiley & Sons, 2011

10. Data Mining: Practical Machine Learning Tools and Techniques,Ian H. Witten, Eibe Frank, Mark A. Hall, Christopher J. Pal,Morgan Kaufmann, 2016

11. Deep Learning Adaptive Computation and Machine Learning series,Ian Goodfellow, Yoshua Bengio, Aaron Courville,MIT Press, 2016

12. Crucial Conversations: Tools for Talking When Stakes are High: Tools for Talking When Stakes Are High,Kerry Patterson, Joseph Grenny, Ron McMillan, Al Switzler,McGraw-Hill Education, 2002

13. The Pragmatic Programmer: Your journey to mastery, 20th Anniversary Edition,Vol-2,David Thomas, Andrew Hunt,Addison-Wesley Professional,2019

14. Recommender Systems Handbook,Editors FrancescoRicci,Lior Rokach, Bracha Shapira, Paul B. Kantor,Springer Science & Business Media, 2010

15. Building Machine Learning Powered Applications: Going from Idea to Product,Emmanuel Ameisen,"O'Reilly Media, Inc.", 2020

9. ANNEXURE

```
from django.shortcuts import render
from django.http import HttpResponseRedirect, HttpResponse,
JsonResponse
from django.contrib.auth import authenticate,logout,login
from django.contrib.auth.decorators import login_required
from django.urls import reverse
from django.db.models import Q
from django.contrib.auth.models import User
from .models import
userProfile,Gigs,Saved,Chat,Message,Question,Option,Question
Answer,InterviewQuestion,InterviewQuestionAnswer,Video

def homepage(request):
    request.session['login_error'] = 0
    if request.user.is_authenticated:

        profile = userProfile.objects.filter(user =
request.user).first()
        if profile.is_company:
            gigs = Gigs.objects.filter(user =
request.user).order_by('-date')
        else:
            gigs = Gigs.objects.all().order_by('-date')

        gigs_list = []
        for gig in gigs:
            save = Saved.objects.filter(Q(gig_pk = gig.pk) & Q(user
= request.user)).first()
```

```
            if save:
                saved = 1
            else:
                saved = 0
            user = User.objects.filter(pk = gig.user.pk).first()
            p = userProfile.objects.filter(user = user).first()
            print("save;",saved )
            dictionary =
{'title':gig.title,"user":gig.user,"image":gig.image.url,"price":gig.
price,"pk":gig.pk,"avatar":p.image.url,"saved":saved,"date":gig.
date}
            gigs_list.append(dictionary)
    else:
        profile = ''
        gigs = Gigs.objects.all()
        gigs_list = []
        for gig in gigs:
  try:
                save = Saved.objects.filter(Q(gig_pk = gig.pk) &
Q(user = request.user)).first()
                if save:
                    saved = 1
                else:
                    saved = 0
            except:
                saved = 0
            user = User.objects.filter(pk = gig.user.pk).first()
            p = userProfile.objects.filter(user = user).first()
            dictionary =
```

```
{'title':gig.title,"user":gig.user,"image":gig.image.url,"price":gig.
price,"pk":gig.pk,"avatar":p.image.url,"saved":saved,"date":gig.
date}
            gigs_list.append(dictionary)

    # all_from_chats = Chat.objects.filter(from_user =
request.user.username)
    # final_from_chats = []
    # print(all_from_chats)
    # for chat in all_from_chats:
    #     # if chat.from_user != request.user.username
  all_chats_to = []
    chats = Chat.objects.filter(to_user = request.user)
    for chat in chats:

all_chats_to.append({"username":chat.from_user,"image":"pro.i
mage.url","message":"mesg.message"})

    chats2 = Chat.objects.filter(from_user = request.user)
    for chat in chats2:

all_chats_to.append({"username":chat.to_user,"image":"pro.ima
ge.url","message":"mesg.message"})

    profiles = userProfile.objects.filter(is_company =
False).order_by('-date')
    profiles_list = []

    for p in profiles:
```

```
        if p.user.username != "guestuser":
            prof =
{"username":p.user.username,"location":p.location,"starting":p.s
tarting_rate,"ending":p.maximum_rate,"skills":p.skills.split(',')[:
2],"image":p.image.url,"pk":p.user.pk,"job":p.job,"is_company":
p.is_company,"age":p.age}
            profiles_list.append(prof)

    return render(request,'index.html',context =
{'profile':profile,"gigs":gigs_list[0:4],'chats':all_chats_to,"profile
s":profiles_list[:4]})


message = 0
reg_error = 0


def homepage1(request):
    return HttpResponseRedirect(reverse('homepage'))
```